UNIVERSITY OF
CAMBRIDGE
Department of Computer
Science and Technology

May 2026

Query-based Dependent Type Elaborator

Submitted in partial fulfilment of the requirements for the
Computer Science Tripos, Part II

Declaration

I, <Author> of <College>, being a candidate for the Computer Science Tripos, Part II, hereby declare that this dissertation and the work described in it are my own work, unaided except as may be specified below, and that the dissertation does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed:
Date: May 2026

Proforma

Candidate Number	TODO
Title	Query-based Dependent Type Elaborator
Examination	Computer Science Tripos — Part II, 2026
Word Count	8,532[1]
Code Line Count	15,774[2]
Project Originator	The candidate
Project Supervisor	Dr Jon Sterling

0.1 Original aims of the project

Dependently typed languages — such as Lean, Agda, and Rocq — require type checkers that execute arbitrary programs at compile time. Most existing implementations operate in batch mode, re-checking entire files after each edit. This project set out to investigate whether a query-based, incremental approach could reduce the latency of elaboration in an interactive setting, by applying the "Build Systems à la Carte" framework to dependent type elaboration.

0.2 Work completed

TODO: fill in once evaluation is complete.

0.3 Special difficulties

None.

[1] Computed via `pdftotext` on the main body (Introduction through Conclusion), including code blocks, tables, and captions.

[2] Computed with `cloc`.

Abstract

Dependently typed languages require expensive type checking, yet most elaborators operate in batch mode, re-checking entire files after each edit. This project implements an incremental, query-based elaborator for a dependently typed language supporting dependent function types, universe polymorphism, and inductive types with recursors. The elaborator is built on a formalisation of the "Build Systems à la Carte" framework in Lean 4, using a Shake-based incremental build system to memoise elaboration queries across edits. We evaluate the system on a standard library and conversion checking benchmarks, comparing incremental re-elaboration against batch re-elaboration.

Acknowledgements

TODO

Contents

Chapter 1

Introduction

1.1 Motivation

A dependently typed elaborator runs programs during compilation. When a definition changes, the elaborator must re-check not just the definitions that mention it, but potentially any definition whose type checking required reducing through its body. In current proof assistants — Lean [1], Agda [2], Rocq [3] — this means re-checking everything from the edit point onward. In an interactive setting, where the user expects rapid feedback on each keystroke, this re-elaboration latency is a bottleneck.

Existing systems offer limited forms of incrementality. Lean, Agda, and Rocq all support *file-level* caching: if a file has not changed, its previously compiled result is reused. Within a single file, Lean 4 saves snapshots of elaboration state and resumes from the most recent valid checkpoint; Agda caches type-checking results per declaration and reuses them when the declaration and its dependencies are unchanged; and coq-lsp re-checks from the first modified sentence onward. All three treat the file as a sequence and reprocess a suffix. If definition 5 of 200 changes, definitions 6 through 200 are re-checked — even if none of them depend on definition 5.

In practice, a user working interactively — editing a lemma, fixing a definition, exploring a proof — expects feedback within a few hundred milliseconds. When an edit triggers re-elaboration of an entire file, the delay can reach tens of seconds in large developments. The user waits, loses context, and must re-orient after each edit.

Most of this re-checking is redundant. If one definition changes but its type is unaffected, definitions that depend only on its type need not be re-checked. We need a way to track which results depend on which, and recompute only what has been invalidated. Unlike in a conventional compiler, where downstream code depends only on type signatures, a dependently typed elaborator may reduce through definition *bodies* during conversion checking. The dependency structure is finer-grained and discovered dynamically.

1.2 Prior art

Query-based incrementality has been applied to conventional compilers: rust-analyzer [4] uses the Salsa framework [5] for incremental type checking of Rust. In these languages, the

compilation boundary is the type signature — changing a function body without changing its type cannot affect downstream checking. Fredriksson's Sixten (later Sixty) [6] applies query-based incrementality to a dependently typed language. It demonstrates viability, but does not formalise the underlying build system or verify that incremental results agree with batch elaboration. No mainstream proof assistant — Lean, Agda, or Rocq — uses query-based incrementality.

The theoretical foundation for fine-grained recomputation is *self-adjusting computation* [7], which tracks data dependencies at runtime and propagates changes through a dependency graph. Adapton [8] refines this with demand-driven evaluation. The "Build Systems à la Carte" framework applies these ideas at the granularity of named build targets rather than individual memory cells.

In a different guise, this is what build systems do. Mokhov et al. [9] show that build systems such as Make, Shake, and Bazel are instances of a single polymorphic type, parameterised by the class of effects a task is allowed to perform. *Shake* supports dynamic dependencies — discovered during execution rather than declared up front — and *early cutoff* — skipping dependents when a recomputed result is unchanged. These are the properties needed: the elaborator discovers which definitions it must unfold only as it runs, and early cutoff prevents unnecessary re-elaboration when an intermediate result has not changed.

1.3 This project

I apply that framework to elaboration. The elaborator decomposes its work into fine-grained queries — "what is the type of this constant?", "what are the errors in this definition?", "what is the parsed AST of this file?" — each memoised by a Shake-based build system. When a definition changes, only the queries that transitively depend on it are reconsidered; if a recomputed result is unchanged, its dependents are not recomputed.

The elaborator supports dependent function types, Tarski-style universes with universe polymorphism, and inductive types with recursors — enough to elaborate a standard library of arithmetic, equality, well-founded recursion, and algebraic structures. The implementation is in Lean 4, and the build system framework is formalised in the same language: three build systems (batch, memoising, and incremental) are proved to agree with batch evaluation.

1.4 Contributions

- A formalisation of the "Build Systems à la Carte" framework in Lean 4, refined with dependent query types (`Val : Key → Type`), a separation of inputs from queries, and a well-foundedness condition on the dependency relation. Three build systems — Busy (batch), LessBusy (memoising), and Shake (incremental) — are proved correct: their results equal batch evaluation by construction.
- An incremental elaborator for a dependently typed language supporting dependent function types, a hierarchy of Tarski-style universes with universe polymorphism, inductive types with recursors, and structures with projections. Elaboration is decomposed into per-declaration queries executed by the formalised build system.
- An evaluation on a standard library and synthetic benchmarks, comparing incremental re-elaboration against batch elaboration and against Lean 4.

1.5 Overview

Section 2 introduces background: the build system framework of Mokhov et al. and our refinements, the dependent type theory we implement, bidirectional type checking, normalisation by evaluation, and related work. Section 3 describes the elaborator's architecture and each pipeline component: parsing, type checking, evaluation, conversion checking, inductive type elaboration, and incremental build and language server infrastructure. Section 4 presents measurements of correctness, batch and incremental performance, and conversion checking speed. Section 5 summarises the results and discusses future work.

Chapter 2

Preparation

2.1 Dependent type theory

We implement a dependent type theory in the tradition of Martin-Löf. We assume familiarity with the simply typed lambda calculus and focus on what distinguishes our system.

2.1.1 Syntax

The core theory maintains a syntactic distinction between *types* and *terms* (Tarski-style universes). The grammar is:

$$\ell ::= n \mid 0 \mid \mathsf{succ}(\ell) \mid \max(\ell, \ell) \qquad \text{(universe levels)}$$

$$A, B ::= \mathsf{Type}_\ell \mid \Pi(x : A).B \mid \mathsf{El}(t) \qquad \text{(types)}$$

$$t, s ::= \mathsf{Type'}_\ell \mid x_i \mid c.\{\overline{\ell}\} \mid \lambda(x : A).t \mid t\, s \mid \Pi'(x : t).s \mid t.k \mid \mathsf{let}\, x : A := t; s \qquad \text{(terms)}$$

Universe levels ℓ form an algebra of named variables n, zero, successor, and binary maximum. Types include universes Type_ℓ, dependent function types $\Pi(x : A).B$, and the decoding operation $\mathsf{El}(t)$ converting a term-level code into a type. The term formers $\mathsf{Type'}_\ell$ and $\Pi'(x : t).s$ are codes for the corresponding type formers. Variables x_i are de Bruijn indices, $c.\{\overline{\ell}\}$ are global constants applied to universe level arguments, and $t.k$ projects the k-th field.

For example, the type `Nat` $\to$ `Nat` is $\Pi(x : \mathsf{El}(\mathsf{Nat}))$. $\mathsf{El}(\mathsf{Nat})$, where Nat is a term-level constant of type Type_0. El decodes the term Nat into the type it represents. Without it, Nat cannot appear where a type is expected — the syntactic categories are disjoint.

In Russell-style theories (as in Lean 4), types and terms share a single syntactic sort: `Nat` can appear directly as a type without coercion. In our Tarski-style formulation, every position in the syntax tree is unambiguously either a type or a term. We prefer Tarski style because the syntactic separation simplifies the metatheory of our system.

2.1.2 Judgement forms

The theory has six judgement forms, parameterised by a global environment Δ and a local context Γ:

$\vdash \Delta\,\mathsf{sig}$	global environment well-formedness
$\Delta; \Gamma \vdash$	context well-formedness
$\Delta; \Gamma \vdash A\,\mathsf{type}$	type well-formedness
$\Delta; \Gamma \vdash t : A$	term typing
$\Delta; \Gamma \vdash A \equiv B\,\mathsf{type}$	judgemental type equality
$\Delta; \Gamma \vdash a \equiv b : A$	judgemental term equality

Δ maps constant names to their definitions and types. Γ is a telescope of typed bindings.

2.1.3 Typing rules

Type formation follows the grammar: Type_ℓ is well-formed in any well-formed context, $\mathsf{El}(t)$ when t has type Type_ℓ, and $\Pi(x : A).B$ when A and B are.

The two central typing rules are introduction and elimination of dependent functions:

$$\frac{\Delta; \Gamma \vdash A\,\mathsf{type} \qquad \Delta; \Gamma, x : A \vdash b : B}{\Delta; \Gamma \vdash \lambda(x : A).b : \Pi(x : A).B}\,\Pi\text{-I} \qquad \frac{\Delta; \Gamma \vdash f : \Pi(x : A).B \qquad \Delta; \Gamma \vdash a : A}{\Delta; \Gamma \vdash f\,a : B[a/x]}\,\Pi\text{-E}$$

Lambdas introduce Π types; application eliminates them, substituting the argument into the codomain. The remaining rules are standard: variables by context lookup, constants by environment lookup, let-bindings by checking the bound value and then the body, and the conversion rule allowing a term of type A to be given type B when A and B are definitionally equal.

2.1.4 Definitional equality

Definitional equality is the least congruence generated by:

- β-reduction: $(\lambda(x : A).b)\,a \equiv b[a/x]$
- δ-reduction: $c.\{\overline{\ell}\} \equiv t \quad$ when $\Delta(c) = t$
- ζ-reduction: $\mathsf{let}\,x : A := a\,\mathsf{in}\,b \equiv b[a/x]$
- η-conversion: $f \equiv \lambda(x : A).f\,x \quad$ at function type
- ι-reduction: recursor applied to a constructor computes to the corresponding branch

The full relation includes congruence, symmetry, transitivity, and closure under conversion for types.

2.1.5 Universe polymorphism

Universes are non-cumulative: Type_ℓ does not belong to $\mathsf{Type}_{\ell'}$ for $\ell' > \ell$. Definitions may be parameterised by universe level variables:

```
def id.{u} (α : Type u) (x : α) : α := x
```

At each use site, levels are instantiated explicitly: `id.{0}` or `id.{v}`. There is no universe level inference.

2.1.6 Inductive types

The theory supports user-defined inductive families. An inductive declaration introduces a type, its constructors, and a *recursor*. We illustrate with `Vector`:

```
inductive Vector.{u} (α : Type u) : Nat → Type u where
  | nil : Vector α Nat.zero
  | cons (n : Nat) (head : α) (tail : Vector α n) : Vector α (Nat.succ n)
```

It generates:

- The type `Vector.{u} : (α : Type u) → Nat → Type u`
- Constructors `Vector.nil.{u} : (α : Type u) → Vector α Nat.zero` and `Vector.cons.{u} : (α : Type u) → (n : Nat) → α → Vector α n → Vector α (Nat.succ n)`
- A recursor `Vector.rec.{v, u}` that eliminates a `Vector` by providing a case for `nil` and a case for `cons`:

```
Vector.rec.{v, u} :
  (α : Type u) →
  (C : (n : Nat) → Vector α n → Type v) →
  C Nat.zero (Vector.nil α) →
  ((n : Nat) → (head : α) → (tail : Vector α n) → C n tail → C (Nat.succ n)
(Vector.cons α n head tail)) →
  (n : Nat) → (xs : Vector α n) → C n xs
```

All computation on inductives proceeds through the recursor; there is no primitive pattern matching. The computational rule (*iota-reduction*) fires when the recursor is applied to a constructor — applying `Vector.rec` to `Vector.cons` reduces to the `cons` branch with the fields and recursive result as arguments.

As an example, `Vector.map` applies a function to each element:

```
def Vector.map.{u, v} (α : Type u) (β : Type v) (n : Nat) (f : α → β)
    : Vector.{u} α n → Vector.{v} β n :=
  Vector.rec.{v, u} α (fun k _ ⇒ Vector.{v} β k)
    (Vector.nil.{v} β)
    (fun m h _ ⇒ Vector.cons.{v} β m (f h))
    n
```

Structures (single-constructor inductives) support *projections* $t.k$, extracting the k-th field from a constructor application.

2.2 Elaboration

The typing rules above are declarative: they specify *what* is well-typed, not *how* to check it. Elaboration is the algorithmic counterpart. We outline the key ideas; implementation details are deferred to Section 3.

2.2.1 Bidirectional type checking

Bidirectional type checking [10] splits the typing judgement into two modes: *checking*, where the expected type is known and pushed into the term, and *inference*, where the type is synthesised bottom-up. The algorithm is syntax-directed — each term form has one applicable rule per mode — and eliminates backtracking. A subsumption rule bridges the two modes via conversion checking.

I omit metavariables and unification; their complexity is orthogonal to incrementality.

2.2.2 Normalisation by evaluation

The conversion rule requires deciding definitional equality, which requires reducing terms. *Normalisation by evaluation* (NbE) [11] evaluates syntax into a semantic domain of values, where equality is checked by structural comparison rather than repeated rewriting. The two operations are *evaluation* (syntax to values) and *quotation* (values back to syntax). NbE performs substitution in bulk through environments rather than one variable at a time, and stops at weak head normal form — reducing only as far as needed.

2.2.3 Glued evaluation

Unfolding every defined constant to its normal form is wasteful: most conversion checks succeed or fail long before full normalisation. *Glued evaluation* [12] pairs each defined constant with a folded form (the constant itself) and an unfolded form (its definition body, available lazily). Conversion checking first compares folded forms; only when heads differ does it force the unfolded forms. For incrementality, this reduces the number of definition bodies the elaborator depends on.

2.2.4 Conversion checking

With NbE and glued evaluation, conversion checking compares two values by structural descent. Following [12], the checker operates in three modes — *rigid*, *flex*, and *full* — controlling how eagerly definitions are unfolded. When two neutrals share a defined head, rigid mode speculatively compares their spines in flex mode, which performs no unfolding at all; if the speculative check fails, rigid falls back to delta-reducing both sides and comparing in full mode, which unfolds eagerly. Each mode that avoids unfolding a definition body avoids creating a dependency on it in the build system — approximate conversion checking directly narrows the dependency graph.

2.3 Incremental computation

The dominant cost in elaboration is conversion checking. Type checking a single definition may trigger many conversion checks, each of which may unfold and normalise arbitrarily large terms. In an interactive setting, most of these checks need not be repeated — only those whose dependencies actually changed. A query-based system can track which definitions each check unfolded, and skip the rest.

2.3.1 Build systems à la carte

Mokhov et al. [9] observe that build systems bring outputs up to date with respect to changed inputs. They capture Make, Shake, Bazel, and others as instances of a single polymorphic type.

The central abstraction is the *task*: a recipe that requests the values of other keys through a callback, with the build system deciding how those requests are fulfilled. A task is polymorphic in an effect — it works with any monad the build system chooses. The constraint on the monad determines what the task can do: `Applicative` means all fetches are declared upfront (static dependencies); `Monad` lets the task inspect a result and decide what to fetch next (dynamic dependencies).

The paper decomposes build systems along two axes: the *scheduler* determines execution order, while the *rebuilder* determines whether a key needs recomputation:

	Topological	Restarting	Suspending
Dirty bit	Make	Excel	-
Verifying traces	Ninja	-	Shake
Constructive traces	CloudBuild	Bazel	-
Deep constr. traces	Buck	-	Nix

Dependent type elaboration needs dynamic dependencies (the elaborator discovers which definitions it must unfold only as it runs) and early cutoff (skipping dependents when a recomputed result is unchanged). This places us at the Shake cell.

The polymorphism of the task type makes correctness modular: the elaborator defines tasks without knowing which build system executes them, and vice versa. The formalisation of this framework in Lean 4, including correctness proofs for three build systems, is presented in Section 4.

2.4 Related work

2.4.1 Efficient elaboration

Kovacs's *smalltt* [12] combines higher-order abstract syntax, glued evaluation, and approximate conversion checking, achieving type-checking speeds that outperform production systems. I adopted approximate conversion checking and glued evaluation from it.

2.4.2 Lean 4

Lean 4 [1] is the primary reference for surface language and inductive types. Our elaborator supports the same declaration forms (`def`, `inductive`, `structure`, `axiom`) and recursor-based elimination. The core theory departs from Lean's kernel in using Tarski-style universes and omitting metavariables and implicit arguments. The metatheory also differs, as we opt for predicative universes, rather than the impredicative Calculus of Constructions.

2.4.3 Self-adjusting computation and Adapton

The theoretical ancestor of fine-grained incremental computation is *self-adjusting computation* [7], which tracks data dependencies at runtime and propagates changes through a dependency graph. Adapton [8] refines this with demand-driven evaluation: computations are re-executed only when their results are demanded. The "Build Systems à la Carte" framework [9] applies these ideas at the granularity of named build targets rather than individual memory cells; our formulation inherits this coarser granularity.

2.4.4 Salsa

Salsa [5] is a Rust framework for on-demand, incrementalised computation, used by rust-analyzer [4] for incremental type checking of Rust. It implements query-based incrementality with memoisation and early cutoff, similar to the Shake cell. I target a dependently typed

language, where conversion checking blurs the boundary between signatures and bodies, and provide a formal correctness proof.

2.4.5 Sixty

Fredriksson's *sixty* [6] applies query-based incrementality to a dependently typed language, using a Haskell library inspired by BSALC. I formalise the build system in Lean 4 with a machine-checked correctness proof, and implement the elaborator in the same language, so the two share types and definitions.

2.5 Development methodology

The elaborator was built in three phases: a batch elaborator for the core theory (Pi types, universes, bidirectional checking), then inductive types and structures, then the incremental query architecture. Each phase was validated against the standard library before moving to the next.

2.5.1 Version control and testing

The codebase is in Git with GitHub as the remote. The dissertation is a separate Typst repository.

The primary correctness test is the standard library: 2,200 lines across 36 files, re-elaborated from scratch on every run. Equality proofs serve as integration tests for the conversion checker. A test harness (`Qdt/Lsp/Test.lean`) simulates editor interactions — setting file contents, triggering rebuilds, asserting on diagnostics and hovers — covering pathological incremental scenarios (swapping definitions, renaming, moving between files). Additional suites include Church-encoded normalisation benchmarks and a port of the Lean 2 HoTT library.

The build system's correctness proofs (Busy, LessBusy, Shake) replace testing of the incremental layer: any inhabitant of `Build` is correct by construction.

2.5.2 Tooling

Development used VSCode with the Lean 4 extension, Lake as the build system, and a C FFI for the performance-critical Shake implementation. Lean's macro system embeds qdt programs directly in Lean source files as inline test cases, since qdt's syntax is a subset of Lean's. The CLI provides `--profile` for per-query timing, `--dump-graph` for exporting the dependency graph, and `--watch` for re-elaboration on save.

2.6 Starting point

I had experience implementing type checkers for simply typed languages, but had not built a dependently typed elaborator or a build system. I was familiar with type theory and had used Lean 4 as a proof assistant, but not as a general-purpose programming language.

No code was written before the project started. The project was initially planned in Rust, but I switched early to OCaml, then to Lean 4. No existing codebases were used as a basis; I consulted smalltt, sixty, and Lean 4′s source as references.

Chapter 3

Implementation

This chapter describes the implementation of the elaborator. We follow the pipeline from source text to elaborated output: parsing, bidirectional type checking, normalisation by evaluation, conversion checking, inductive type elaboration, and the incremental build architecture that ties them together.

The implementation is written in Lean 4. Lean is designed as a general-purpose programming language with a native C backend and C FFI, which we use for the performance-critical build system implementations. Its macro system lets us embed qdt programs directly in Lean source files: since qdt's syntax is a subset of Lean's, inline test cases are Lean macros that produce qdt source strings. Lean's dependent types make terms intrinsically scoped (the `n` in `VTm n`) and allow query result types to depend on the query key (`Val : Key → Type`). The build system's correctness proofs live in the same language as the implementation — the `Build` type carries its proof, and the elaborator's tasks are the same object the proofs reason about.

3.1 Architecture

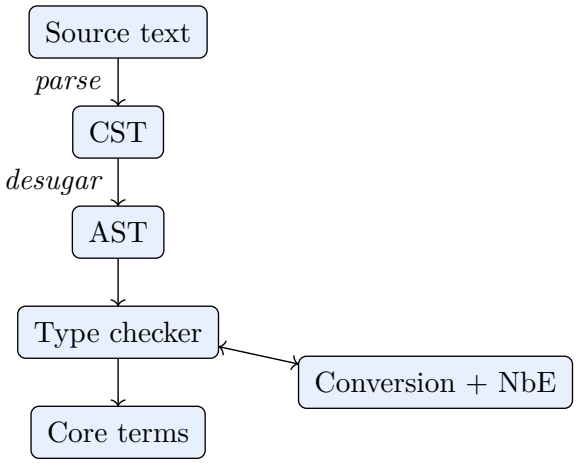

Figure 1: Elaboration pipeline. Source text is parsed to a CST, then desugared to an AST. The type checker produces core terms, calling the conversion checker and NbE evaluator as subroutines. Each stage is a query memoised by the build system.

Each stage of the pipeline (Figure 1) is described in a separate section:

- **Parsing** (Section 3.2): a Pratt parser produces a green-tree CST and desugared AST. Green trees ensure whitespace edits do not invalidate downstream queries.
- **Bidirectional type checking** (Section 3.3): the AST is checked against expected types or synthesises its own, calling the evaluator and conversion checker.
- **Normalisation by evaluation** (Section 3.4): terms are evaluated into a semantic domain using defunctionalised closures and de Bruijn levels.
- **Conversion checking** (Section 3.5): decides definitional equality with rigid, flex, and full modes to avoid unnecessary unfolding.
- **Inductive types** (Section 3.6): declaratoins are elaborated into type formers, constructors, and recursors with iota-reduction rules.
- **Incremental elaboration** (Section 3.7): the build system decomposes elaboration into per-declaration queries, memoises results, and recomputes only what changed.
- **Language server** (Section 3.8): diagnostics and hover information served to a VSCode extension.

The elaboration monad and query system are described in Section 3.7.

Components are connected by query dependencies, not direct function calls. The parser produces a `cst` query result; the desugarer an `ast` query; each declaration's elaborated form an `elabDecl` query depending on `constant` queries for every name it references. When a file changes, only the affected queries are recomputed.

3.1.1 Worked example

We trace the elaboration of two definitions:

```
def id.{u} (A : Type u) (x : A) : A := x
def foo : Type 1 := id.{2} (Type 1) Type
```

Parsing. The source is parsed into a green-tree CST, then desugared to an AST with two definition nodes at indices 0 and 1. The `declarationIndex` query maps the name `id` to index 0 and `foo` to index 1.

Elaborating `id`. The `elabDecl` query for `id` fires. The universe parameter list is `[u]`. The parameter telescope is elaborated left to right:

- `A : Type u` — the annotation is a valid universe, so `A` is bound with type Type_u.
- `x : A` — the annotation refers to the parameter just bound. After evaluation, `x` is bound with type A.

The return type `A` refers to the first parameter. The body `x` is checked against `A` by context lookup. The elaborated constant has type $\Pi(A : \text{Type}_u).\Pi(x : A).A$ and body $\lambda A.\lambda x.x$.

Elaborating `foo`. The `elabDecl` query for `foo` fires. The expected type Type_1 is checked (it has type Type_2). The body `id.{2} (Type 1) Type` is an application, handled by `inferTm`:

1. **Infer the head.** `id` triggers `fetchConstant`, issuing a `constant` query and recording a dependency. The type is instantiated at level 2: $\Pi(A : \text{Type}_2).\Pi(x : A).A$. Evaluation produces a glued value: a neutral head `.const id [2]` paired with name and universe arguments for deferred unfolding.

2. **Check the first argument.** `Type 1` is checked against Type_2. Inferred type: Type_2. Conversion: $\text{Type}_2 = \text{Type}_2$ — success. The codomain closure is evaluated with A bound to Type_1, yielding $\Pi(x : \text{Type}_1).\text{Type}_1$.
3. **Check the second argument.** `Type` (Type_0) is checked against Type_1. Inferred: Type_1. Conversion: $\text{Type}_1 = \text{Type}_1$ — success.
4. **Subsumption.** Inferred result Type_1 against expected Type_1. Conversion succeeds.

No delta-reduction of `id` was needed — the result type follows entirely from `id`'s Pi type, without inspecting its body. The glued value was never forced.

Query dependencies. `elabDecl foo` depends on `constant id`. If `id`'s body later changes but its type remains $\Pi(A : \text{Type}_u).\Pi(x : A).A$, the hash is unchanged and `foo` is not recomputed. Only if `id`'s type changes does `foo` re-fire.

3.2 Parsing

The parser is a hand-rolled recursive descent Pratt parser [13], implemented as monadic combinators over `StateT State (Except ParseError)`. Source text is first parsed to a concrete syntax tree (`Cst`) retaining trivia and exact token widths, then desugared to an abstract syntax tree (`Ast`) that strips trivia and expands surface syntax.

The parser itself is unremarkable . The interesting choice is the CST representation.

3.2.1 Green trees

The CST follows the *green tree* pattern [4], [14]. Nodes and tokens are tagged by `SyntaxNodeKind`; positions are not stored:

```
inductive Cst : Type
  | token (kind : SyntaxNodeKind) (val : String)
  | node (kind : SyntaxNodeKind) (children : Array Cst)
with
  @[computed_field]
  width : Cst → Nat
    | .token _ val ⇒ val.length
    | .node _ children ⇒ children.map width ▷.sum
```

For example, `def foo := Type` produces the tree:

```
node Command.definition ["def " | "foo" | " := " | "Type"]
```

Each token stores its text including adjacent whitespace. No absolute positions — these are recovered by summing widths from the root. Because nodes lack positions, identical subtrees are structurally equal regardless of location, making the CST `Hashable`. The build system can short-circuit recomputation when an edit does not affect a subtree.

If two spaces are inserted at the start of the file, only the first token changes (`"def "` becomes `"  def "`). The rest are structurally identical, so the AST (which discards trivia) hashes the same and no downstream query is invalidated. Without green trees, any whitespace edit would shift absolute positions throughout the file, invalidating every query.

The pipeline works with AST *paths* (lists of child indices from the root), not source positions. Diagnostics and hovers are keyed by path, so elaboration results are independent of where a definition sits in the file. Positions are reconstructed on demand for the language server.

3.2.2 Language server integration

Alongside the CST, parsing produces a `SourceMap` that bidirectionally maps paths in the CST to paths in the AST. A *path* is a list of child indices from the root. In the tree above, the token `"foo"` has path `[1]` (the second child of the root node), and `" := "` has path `[2]`. In a file with two definitions, the identifier in the second definition might have path `[1, 1]` (second child of root, then second child of that node). The bidirectional map is:

```
structure SourceMap where
  cstToAst : HashMap Path Path
  astToCst : HashMap Path Path
```

The map is populated during desugaring. The language

The language server uses it in both directions:

- *Hover*: walk the CST to find the path under the cursor, then `cstToAst` gives the AST path. Hover content is keyed by AST path.
- *Diagnostics*: recorded at CST paths, mapped back via `astToCst`, then converted to source spans by summing widths.

Path-based lookup is $O(\text{depth})$ rather than $O(\text{size})$.

3.2.3 Error recovery

On a parse error, a diagnostic is emitted and the parser skips to the next top-level declaration boundary (`def`, `inductive`, `axiom`, etc.). A single malformed declaration does not prevent the rest of the file from being elaborated.

A separate converter from `Lean.Syntax` to `Ast` via the metaprogramming framework is available for inline test cases in Lean source files, but is not used by the main pipeline.

3.3 Bidirectional type checking

Type checking is bidirectional [10]. Rather than a single judgement $\Gamma \vdash t : A$, the algorithm splits into two mutually recursive functions:

- `inferTm ctx ast : OptionT ElabM (Tm n × VTy n)` — synthesise a core term and its type. Used when the expected type is unknown.
- `checkTm ctx expectedTy ast : ElabM (Tm n)` — produce a core term at a known expected type. On failure, a diagnostic is emitted and a fresh axiom inserted as placeholder.

The split means a lambda's parameter type comes from the expected Pi type (so it can be left unannotated), the algorithm is syntax-directed (one case per AST constructor), and mismatches are reported at the exact subterm.

The mode-switching rules:

- **Lambdas** are checked against $\Pi(x : A).B$: the body is checked against B with $x : A$ in context. If the parameter has a type annotation, it must match A; otherwise A is taken from the expected type.
- **Applications** are inferred. The function f is inferred to type T. If T is not Pi after whnf, the elaborator fails. Otherwise $T = \Pi(x : A).B$, the argument a is checked against A, and the result has type $B[a/x]$.
- **Variables** are inferred by context or environment lookup.
- **Let-bindings**: infer the bound value's type, check the body in the extended context.
- **Universes** Type_ℓ are inferred to have type $\mathsf{Type}_{\ell+1}$.
- **Subsumption**: when inference meets a checking position, the inferred type is compared against the expected type by the conversion checker.

`inferTm` and `checkTm` are mutually recursive with `checkIdent` for variable lookup (local context first, then global environment). The application case of `inferTm`:

```
| .node `Term.app #[fn, arg] ⇒ do
    let (fnTm, fnTy) ← inferTm ctx fn
    let .pi _ dom ⟨env, codTm⟩ := fnTy
      | raiseError (.expectedFunctionType ctx.names (← fnTy.quote))
    let argTm ← checkTm ctx dom arg
    let argVal ← argTm.eval ctx.env
    let codVal ← codTm.eval (env.cons argVal)
    return (.app fnTm argTm, codVal)
```

`inferTm` always returns a `VTy` produced by `Ty.eval`, so `fnTy` is already in weak head normal form — no `whnf` call is needed to expose the Pi. The codomain is substituted via evaluation in the closure's extended environment, not explicit syntactic substitution.

On failure — `sorry`, unbound variable, type mismatch — the elaborator emits a diagnostic and inserts a fresh axiom as placeholder. The rest of the file remains type-checkable, so the language server can report errors throughout.

3.4 Normalisation by evaluation

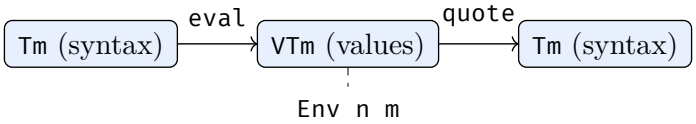

Figure 2: NbE evaluates syntax into values via an environment, and quotes values back to syntax.

3.4.1 Semantic domain

Values (`VTm`, `VTy`) represent terms in weak head normal form, using de Bruijn *levels* [15] rather than indices:

```
inductive VTm : Nat → Type
  | u'      : Universe → VTm n
  | neutral : Neutral n → VTm n
  | lam     : Name → VTy n → ClosTm n → VTm n
```

```
  | pi'     : Name → VTm n → ClosTm n → VTm n
  | glued   : Neutral n → Name → List Universe → VTm n
```

A `neutral` is a head (variable or constant) applied to a spine of eliminators. A `lam` carries a closure — an `Env n m` paired with a `Tm (m + 1)`. Beta-reduction evaluates the body in the extended environment. A `glued` value carries the folded neutral form (for quotation) alongside the constant's name and universe arguments; unfolding happens on demand when the conversion checker or whnf forces it.

A de Bruijn *index* counts inward from the binding site: in $\lambda x.\lambda y.x$, x has index 1. A de Bruijn *level* counts outward from the root: x has level 0. Indices shift when the context grows; levels do not.

NbE frequently *weakens* values — embedding them into a larger scope when a closure is opened. Under indices, weakening traverses the entire value to shift every variable. Under levels, it is a no-op. The scope parameter `n` in `VTm n` enforces well-scoping at the type level; weakening becomes a proof obligation that `omega` discharges, and the data is reused via `unsafeCast`.

3.4.2 Closures

Closures are defunctionalised: a lambda carries its body as syntax paired with the captured environment, rather than a host-language function `VTm → VTm` (HOAS). HOAS is faster [12] but requires a non-strictly-positive inductive (`lam : (VTm → VTm) → VTm`), which Lean's kernel rejects. Defunctionalisation keeps the evaluator within the logic.

3.4.3 Evaluation

`Tm.eval` interprets syntax in an environment of values, indexed by scope size:

```
partial def Tm.eval {n c} : Tm c → SemM n c (VTm n)
  | .u' i         ⇒ return .u' i
  | .var i        ⇒ return (← read).get i
  | .const name us ⇒ do
      let some _ ← fetchConstantInfo name
        | return .neutral ⟨.const name us, .nil⟩
      if (← fetchDefinition name).isSome then
        return .glued ⟨.const name us, .nil⟩ name us
      else
        return .neutral ⟨.const name us, .nil⟩
  | .lam x a body ⇒ return .lam x (← a.eval) ⟨← read, body⟩
  | .app fn arg   ⇒ do (← fn.eval).app (← arg.eval)
  | .pi' x a b    ⇒ return .pi' x (← a.eval) ⟨← read, b⟩
  | .proj i t     ⇒ do (← t.eval).proj i
  | .letE _ _ t b ⇒ do b.eval (.cons (← t.eval) (← read))
```

Constants evaluate to *glued* values when a definition exists, carrying the folded neutral alongside name and universe arguments — no body is fetched. Axioms and opaques produce plain neutrals. Applications dispatch via `VTm.app`: beta-reduction for lambdas, spine extension for neutrals and glued values.

3.4.4 Weak head normalisation

`VTm.whnf` reduces until the outermost form is canonical (lambda, Pi, universe) or irreducible (variable-headed neutral):

- **δ-reduction**: when the head is a defined constant, `whnf` fetches the definition body, evaluates it with the appropriate universe substitution, and re-applies the spine. This is the only point at which a body is materialised — evaluation merely records name and universes in the glued value.
- **ι-reduction**: when a fully-applied recursor has a constructor as its major premise, `whnf` substitutes the constructor's fields into the minor premise.

A glued value that is never forced — because the conversion checker matched it in flex mode — never triggers a fetch of the definition body.

3.4.5 Quotation

`VTm.quote` converts values back to syntax. Closures are applied to a fresh variable at the current level and the result quoted, converting levels to indices. Glued nodes quote to the folded form (the neutral), preserving definition names rather than inlining bodies.

3.5 Conversion checking

Conversion checking decides definitional equality. The naive algorithm — reduce both sides to normal form, compare structurally — forces all reducible subterms regardless of whether comparison needs them.

The algorithm here, from Kovacs's smalltt [12], bounds speculative work with a three-state mode:

- **Rigid**: the default. When two values have the same defined head, spines are compared speculatively in flex mode. If flex succeeds, no unfolding was needed. If it fails, both sides are unfolded and compared in full mode.
- **Flex**: entered from rigid's speculative check. No definitions are unfolded; no fallback. Any mismatch causes immediate failure, returning control to rigid.
- **Full**: entered after a flex failure. All definitions are unfolded on encounter. Once in full, all recursive calls remain in full.

Figure 3 shows the transitions between these modes. At most one backtrack per subterm: rigid attempts a cheap flex check, and on failure commits to the expensive full check. Flex never unfolds, so the cost of a failed speculation is bounded by the matching prefix of the two spines.

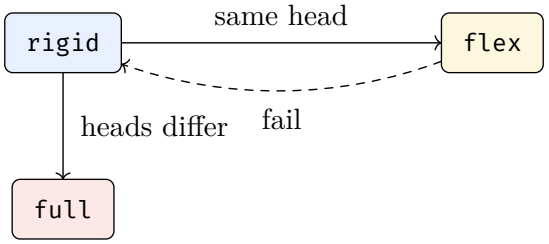

Figure 3: Conversion state transitions. Rigid speculatively tries flex on matching heads; on failure it falls back to full, which unfolds eagerly.

Consider comparing `f (g (h x))` with itself, where `f`, `g`, `h` are top-level definitions. The naive algorithm unfolds all three on both sides — potentially exponential. The approximate algorithm observes the same folded structure: same head `f`, same spine. Flex descends into the argument, sees `g`, then `h`, bottoming out at `x` with no unfolding. The check runs in time proportional to the folded term, not the unfolded one.

For eta-conversion: function eta opens both sides at a fresh variable and compares bodies; structure eta compares each projection $s.i$ with the corresponding field r_i of $c(r_1, ..., r_k)$.

An alternative is *on-the-fly* reduction: whnf each side just enough to decide equality, recurse under binders. This avoids full normal forms, but without the rigid/flex distinction it still unfolds aggressively whenever heads differ syntactically. The speculative flex check avoids this in the common case of matching defined heads.

3.6 Inductive types

An inductive declaration [16] introduces a type former, its constructors, and a recursor. The elaboration proceeds in phases: elaborate the type and parameters, register the inductive as opaque (so constructors can refer to it), elaborate each constructor, check strict positivity and universe consistency, then generate the recursor type and its iota-reduction rules.

For `Nat`, the generated recursor is:

$$\mathsf{Nat.rec}.\{u\} : (C : \mathsf{Nat} \to \mathsf{Type}_u) \to C\,\mathsf{zero} \to ((n : \mathsf{Nat}) \to C\,n \to C\,(\mathsf{succ}\,n)) \to (n : \mathsf{Nat}) \to C\,n$$

with reduction rules:

$$\mathsf{Nat.rec}\,C\,z\,s\,\mathsf{zero} \rightsquigarrow z$$
$$\mathsf{Nat.rec}\,C\,z\,s\,(\mathsf{succ}\,n) \rightsquigarrow s\,n\,(\mathsf{Nat.rec}\,C\,z\,s\,n)$$

There is no primitive pattern matching — all computation on inductives proceeds through the recursor. Structures (single-constructor inductives) additionally generate projection functions and an eta rule: `c (s.f_1) ... (s.f_k)` is identified with `s`.

The motive's universe u is a fresh universe parameter, distinct from the inductive's own universe parameters. This allows elimination into any universe: `Nat.rec.{0}` gives recursion into `Type 0`, while `Nat.rec.{1}` gives large elimination into `Type 1`.

3.7 Incremental elaboration

Elaboration is decomposed into queries managed by a Shake-based build system. Each query has a tag and parameters; its result type is determined by the tag. `Key` has 17 cases; the principal ones:

```
inductive Key where
  | cst : FilePath → Key
  | ast : FilePath → Key
  | declarationIndex : FilePath → Key
  | elabCmdAt : FilePath → Nat → Key
  | elabDecl : FilePath → Name → Key
  | constant : FilePath → Name → Key
  | lookupInfo : FilePath → Name → Key
  -- ... others omitted
```

```
def Val : Key → Type
  | .cst _               ⇒ Cst × Array ParseError
  | .ast _               ⇒ Ast
  | .declarationIndex _  ⇒ HashMap Name Nat × Array Diagnostic
  | .elabCmdAt _ _       ⇒ Global × ElabInfo
  | .elabDecl _ _        ⇒ Option (Constant × Origin) × ElabInfo
  | .constant _ _        ⇒ Option (Constant × Origin)
  | .lookupInfo _ _      ⇒ ElabInfo
  -- ... others omitted
```

The principal queries:

- `declarationIndex`: maps declaration names to indices within the file. Depends only on the parsed AST.
- `elabDecl`: elaborates one declaration. Fetches the AST, looks up the declaration, type-checks it. Depends on `constant` queries for every referenced name.
- `constant`: the elaborated form of a named constant — the primary cross-declaration dependency.

When the elaborator encounters a constant reference, `fetchConstant` issues a `Key.constant` query, registering a dependency. An `entryCache` in `ElabState` memoises lookups within a single elaboration run to avoid repeated queries for the same name.

3.7.1 The elaboration monad

`ElabM` combines incremental queries with elaboration state:

```
abbrev ElabM := ReaderT ElabContext (StateT ElabState (Task Monad config ι₀
q₀))
```

`Task` is the query monad: `Task.fetch` calls register dependencies. `StateT ElabState` threads the local environment, constant cache, and accumulated diagnostics and hovers. `ReaderT ElabContext` carries file path, declaration name, and universe parameters. `ι₀` and `q₀` identify the current query in the dependency graph.

3.7.2 Granularity of queries

If queries are too coarse, such as per-file granuality, any edit invalidates the whole file. If they are too fine, per-expression, the per-query overhead dominates.

I chose per-declaration granularity. `Key.elabDecl` elaborates one `def`, `inductive`, or `structure`. Dependent declarations are recomputed only if the edit changes the cached `Constant`.

Finer granularity is not particularly compelling, since large terms can be broken into smaller definitions anyway, by the user.

3.7.3 Dependently-typed query results

The dependent `Val : Key → Type` lets `fetch (Key.constant p n) : Task (Option Constant)` type-check without tagging — the compiler specialises each call site. The alternatives are a sum type (boilerplate matching at every consumer), an existential (`unsafeCast` at every fetch), or

GADTs (where an index on the key type determines the result type, as in Rock and Salsa). Dependent types express this directly.

3.7.4 Query dependency graph

Figure 4 shows the query dependency graph for a two-definition file:

```
def foo : Type 1 := Type
def bar : Type 1 := foo
```

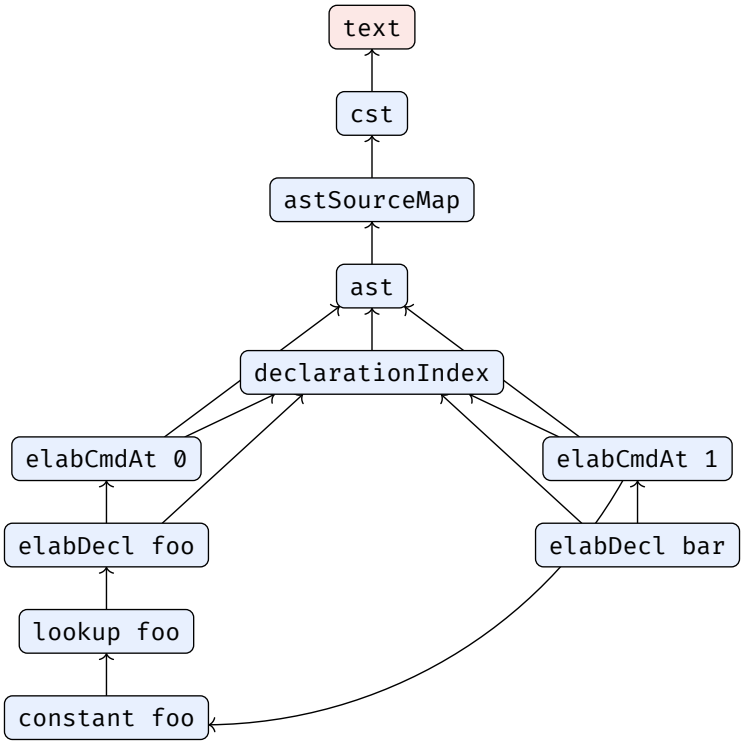

Figure 4: Query dependency graph for a file with two definitions. Arrows point from a query to its dependencies. The `text` node (red) is the input. Elaborating `bar` depends on `constant foo`, which depends on `elabDecl foo`. Editing `foo`'s body invalidates `elabCmdAt 0`, `elabDecl foo`, `lookup foo`, and `constant foo`; if the resulting `Constant` hashes the same (e.g. the type is unchanged), `elabDecl bar` is not recomputed.

3.7.5 Glued evaluation and dependency tracking

Glued evaluation (Section 3.4) produces values carrying a constant's name and universe arguments without fetching its body. The body is fetched only when whnf forces the value. Flex mode compares two glued values with the same head without forcing either side — no body fetched, no dependency recorded. Only when heads disagree and full mode fires does delta-reduction fetch the body and record a dependency.

Editing a definition's body thus invalidates only the queries that actually unfolded it. Call sites whose checks succeeded in flex mode have no dependency on the body.

3.8 Language server

The elaborator doubles as a language server. Hover information and diagnostics are collected in `ElabState` during elaboration and served to a VSCode extension via LSP:

- **Diagnostics**: type errors, unbound variables, universe mismatches — positioned via path indices into the AST.
- **Hover**: the type under the cursor, or the full signature of a referenced constant.

3.8.1 Incremental test harness

The test suite verifies incremental correctness via a harness simulating editor interactions:

- `setText text filepath` sets the contents of a file and triggers a full rebuild through the query system.
- `diagnostics check filepath` asserts that the diagnostics for a file satisfy a predicate.
- `hover pos expected start stop` asserts that hovering at a given position produces the expected content and source span.

Tests sequence multiple `setText` calls and check that diagnostics and hovers update correctly. The cases cover pathological incremental scenarios: swapping definitions so forward references become backward; renaming and checking that dependents report unbound variables; atomically moving a definition between files; and cyclic imports. These exercise dependency tracking, cache invalidation, and error recovery paths that batch elaboration would not stress.

3.9 Repository overview

Directory	Purpose	Lines	Section
`Incremental/`	Build system framework (12 files): Task, Build, BuildConfig, Busy, LessBusy, Shake, Salsa, free theorem, plus ShakeCPS/ShakeEState/ShakeRdeps variants	1,209	2.3
`Incremental/c/`	C implementations of Shake and Salsa (2 files)	1,141	3.7
`Qdt/Frontend/`	Parser, CST, AST, desugarer (4 files)	1,371	3.2
`Qdt/Theory/`	Declarative type theory: syntax, contexts, typing rules, universes, substitution, weakening, alpha-equality (11 files)	1,907	2.1
`Qdt/`	Elaborator core (21 files): bidirectional checker, NbE, conversion, inductive/structure elaboration, quoting, control monad, query types, task rules, error handling, pretty printing, CLI	2,804	3.3–3.7
`Qdt/Test/`	Elaborator regression tests (12 files)	610	4.1
`Qdt/Lsp/Test/`	Incremental test harness + 17 test cases	438	3.8
`FSWatch/`	File system watcher for watch mode (4 files)	361	—
`Main.lean`	CLI entry point	173	—
`Lsp.lean`	Language server entry point	330	3.8

Table 1: Source code by directory. Line counts computed with `wc -l`, excluding `.lake` and stdlib examples. Total: 9,203 lines of Lean + 1,141 lines of C.

Chapter 4

Evaluation

We evaluate correctness, performance, and incrementality.

4.1 Correctness

The primary test is the standard library: 2,200 lines of qdt code across 36 files, covering natural number arithmetic, propositional equality, well-founded recursion, sigma types, monadic abstractions, and the Ackermann function, via well-founded recursion.

Equality proofs exercise the conversion checker: `Eq.refl.{0} Nat 6` at type `Nat.add 2 4 = 6` succeeds only if the elaborator correctly reduces `Nat.add 2 4` to `6` via iota-reduction.

4.2 Batch elaboration performance

TODO: re-measure on final code with hardware spec.

4.3 Conversion checking performance

We stress-test the evaluator and conversion checker with Church-encoded benchmarks from Kovacs's normalisation-bench [12]. These encode natural numbers as lambda terms, then check definitional equality between two independently computed values of the same normal form, forcing n beta-reductions on each side.

Benchmark	Time (ms)
Nat 10K	176
Nat 100K	1,708
Nat 1M	16,417

Table 2: Church numeral conversion checking. Time scales linearly with n.

For comparison, smalltt [12] handles Nat 5M in 90ms using HOAS closures compiled by GHC. The constant-factor gap is due to our defunctionalised closure representation: each beta-reduction re-interprets the body in an extended environment, whereas HOAS compiles the closure to a native function call.

4.4 Scaling benchmarks

We compare elaboration-only time (wall clock minus startup) between qdt and Lean 4 on synthetic programs of increasing size. Lean's measurement includes kernel checking, which qdt does not perform. Startup is measured by elaborating a file containing only an `inductive Nat` definition (qdt: 73ms, Lean: 310ms) and subtracted from all timings.

Benchmark	N	qdt (ms)	Lean (ms)	Ratio
Discrete	1,000	129	460	3.6 ×
Discrete	5,000	896	2,599	2.9 ×
Discrete	10,000	2,713	4,993	1.8 ×
Random	1,000	249	692	2.8 ×
Random	5,000	1,816	3,578	2.0 ×
Random	10,000	5,931	7,535	1.3 ×

Table 3: Elaboration-only time (startup subtracted). Lean's time includes kernel checking. Ratio is Lean/qdt.

Discrete ($n_i := \mathsf{Nat.zero}$, independent definitions) isolates per-definition overhead. *Random* ($n_i := \mathsf{Nat.succ}\, n_j$ for random $j < i$) has a realistic dependency shape. qdt is faster at every size. The gap narrows at larger N, consistent with qdt's per-query overhead being amortised as definitions become more numerous.

4.5 Formal verification

The build system framework is formalised in Lean 4 with machine-checked proofs. We refine the "Build Systems à la Carte" framework [9] with dependent types, separated inputs and queries, and a well-foundedness condition.

4.5.1 Formulation

The parameters are bundled into a configuration:

```
structure BuildConfig : Type 1 where
  I : Type; V : I → Type
  Q : Type; R : Q → Type
  rel : (∀ i, V i) → Q → Q → Prop
  wf : ∀ ι, WellFounded (rel ι)
```

Inputs (`I`, `V`) and queries (`Q`, `R`) are separated, with result types depending on the key — different queries can return different types. The relation `rel` encodes the dependency order, indexed by the input state. It must be well-founded for any input.

The task type refines the original: inputs and queries have distinct callbacks, result types are dependent, and fetch requires a well-foundedness proof:

```
def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f],
    (∀ i, f (𝒞.V i)) →
```

```
    (∀ q, 𝒞.rel ι₀ q q₀ → f (𝒞.R q)) →
    f α
```

The task receives two callbacks: `input` reads an input value `𝒞.V i`, and `fetch` retrieves a query result `𝒞.R q`. The fetch callback requires a proof that `q` precedes `q₀` in the dependency relation, making termination provable by well-founded recursion.

4.5.2 Correctness

A correct build system produces the same result as `compute`, a reference semantics defined by well-founded recursion. `compute` runs each task in `Id`, recursively computing every dependency from scratch — exponential time, serving as a specification:

```
def compute (tasks : Tasks c 𝒞) (ι : ∀ i, 𝒞.V i) (q : 𝒞.Q) : 𝒞.R q :=
  tasks q Id ι (fun q' _ ⇒ compute tasks ι q')
termination_by 𝒞.wf.wrap q
```

A build system carries private state `σ` and returns a value with a proof that it equals `compute`:

```
structure Build (c) (𝒞 : BuildConfig) (J : Type) [Input 𝒞 J] where
  σ : Type
  init : J → σ
  build :
    (tasks : Tasks c 𝒞) → (q : 𝒞.Q) → (store : σ) →
      { r : 𝒞.R q // r = compute tasks (inputs store) q } × σ
```

Any inhabitant is correct by construction. Three build systems inhabit this type:

Busy calls `compute` directly. Correctness is by definition — the returned value is `compute`'s output.

```
def Busy : Build c 𝒞 J where
  build tasks q store :=
    (⟨compute tasks (Input.get store) q, rfl⟩, store)
```

4.5.3 The free theorem

The correctness proofs of LessBusy and Shake both rely on a *free theorem* for tasks, arising from relational parametricity [17], [18], [19], [20].

A `Task` is polymorphic in `f` — it works with any monad satisfying `c` and cannot observe which monad it runs in. To formalise this, we define a `MonadAction` relating two monads: a family of relations `rel` on their carrier types, preserving `pure` and `>>=`:

```
structure MonadAction (κ₁ κ₂ : Type → Type) [Monad κ₁] [Monad κ₂] where
  rel : (α → β → Prop) → κ₁ α → κ₂ β → Prop
  rel_pure : R a b → rel R (pure a) (pure b)
  rel_bind : rel R ma mb → (∀ a b, R a b → rel S (ka a) (kb b))
    → rel S (ma >>= ka) (mb >>= kb)
```

The free theorem then states: if inputs and fetches are pointwise related, the task result is related:

```
axiom Task.freeTheorem :
    (hι : ∀ i, F.rel Eq (ι₁ i) (ι₂ i)) →
    (hfetch : ∀ q hq, F.rel Eq (fetch₁ q hq) (fetch₂ q hq)) →
    F.rel Eq (t κ₁ ι₁ fetch₁) (t κ₂ ι₂ fetch₂)
```

Parametricity cannot be proved internally in Lean; it is stated as an axiom. The axiom is incompatible with `Classical.choice`; the formalisation avoids choice throughout.

From the axiom, we derive `Tasks.freeTheorem`: specialising to `κ₁ := StateM Cache` and `κ₂ := Id`, if the cache provides the same values as `compute` for every fetch, the overall task result equals `compute`. The proof unfolds `compute` once and applies the axiom.

4.5.4 LessBusy

LessBusy memoises intermediate results within a single build. Each cache entry is a dependent pair `{ r : 𝒞.R q // r = compute tasks ι q }` — a value with its correctness proof. The `fetch` function checks the cache first; on a miss, it runs the task, caches the result, and returns it:

```
def fetch (q₀ : 𝒞.Q) :
    StateM VCache (Value q₀) := do
  if let some v := (← get).get? q₀ then return v
  let v ← run q₀ (fun q hq ⇒ fetch q)
  modify (·.insert q₀ v)
  return v
termination_by 𝒞.wf.wrap q₀
```

Termination follows from the well-founded relation: each recursive `fetch` call provides a proof `𝒞.rel q' q₀`, so the recursion decreases. The `run` function executes the task and produces a proven `Value` — this is where the free theorem is used.

The proof constructs a concrete `MonadAction (StateM VCache) Id`:

```
def action : MonadAction (StateM VCache) Id where
  rel P m b := ∀ s, P (m s).1 b
```

This says: a stateful computation `m` is related to a pure value `b` when, from any starting state, `m`'s return value satisfies the relation with `b`. Each cached fetch returns the same value as `compute` (the cache entries carry their own proofs), so `Tasks.freeTheorem` gives the overall result.

The top-level definition ties `fetch` into the `Build` type:

```
def LessBusy : Build Monad 𝒞 J where
  build tasks q store :=
    let ι₀ := Input.get store
    let (v, _) := fetch tasks ι₀ q (DHashMap.empty)
    (v, store)
```

The empty cache is passed as the initial state. Each `fetch` call populates it, and the returned `v` carries a proof that it equals `compute`.

4.5.5 Shake

Shake extends memoisation across builds by storing fingerprints of each query's dependencies. Each cached `Memo` carries a universally quantified invariant:

```
structure Memo (q : 𝒞.Q) where
  value : 𝒞.R q
  inputDeps : List ((i : 𝒞.I) × H)
  deps : List (Σ' (q' : 𝒞.Q) (_ : 𝒞.rel q' q), H)
  invariant :
    ∀ (ι : ∀ i, 𝒞.V i),
      (∀ p ∈ inputDeps, hI p.1 (ι p.1) = p.2) →
      (∀ p ∈ deps, hR p.1 (compute tasks ι p.1) = p.2.2) →
      value = compute tasks ι q
```

The invariant says: for *any* input function `ι`, if every recorded input fingerprint matches `ι` and every recorded dependency fingerprint matches `compute` under `ι`, then `value` equals `compute tasks ι q`. This is universally quantified over `ι` — the same memo is valid across builds with different inputs, as long as the fingerprints match.

On a cache hit, `verifyInputs` checks each recorded input fingerprint against the current inputs, and `verifyDeps` recursively fetches each dependency (getting a proven `Value`) and checks its fingerprint. If both pass, the memo's `invariant` directly gives the correctness proof.

On a cache miss, `runRecompute` evaluates the task via a free monad tree and builds a fresh `Memo`. The invariant proof uses `FM.evalTree_cross`: if two sets of inputs and dependencies agree at every position recorded in the trace (checked via injective embeddings `hI : 𝒞.V i ↪ H` and `hR : 𝒞.R q ↪ H`), the free monad tree evaluates to the same result. The injectivity of the hash functions (`Function.Embedding`) is what makes fingerprint comparison sound: `hash a = hash b` implies `a = b`.

The elaborator's tasks are a single `Tasks` value, independent of which `Build` executes them — incremental elaboration is provably equivalent to `compute`.

The formalisation comprises approximately 1,500 lines of Lean 4 across the core library (Basic, Busy, LessBusy, Shake, FreeTheorem, FreeMonad).

4.6 Incremental re-elaboration

We measure re-elaboration time after targeted edits on the standard library, using a retained Shake store. The Shake build system is instrumented to count cache hits (queries whose fingerprints match and are reused) and recomputed queries. After a cold build populates the store, each edit modifies one input and triggers a rebuild against the existing cache.

Edit	Time (ms)	Speedup
Cold build (no cache)	1,027	1 ×
No-op (retained store, no changes)	129	8.0 ×
Whitespace (append newline to entry file)	127	8.1 ×
Leaf (append definition to Ackermann.qdt)	172	6.0 ×
Hub (append definition to Nat.qdt)	170	6.0 ×

Table 4: Incremental re-elaboration of the standard library after targeted edits. Speedup is relative to cold build.

The no-op rebuild verifies every fingerprint in the store but recomputes nothing — the 129ms is the cost of traversing the dependency graph and checking hashes. The whitespace edit has the same cost: the green tree representation absorbs the whitespace change, the AST hashes the same, and no downstream query is invalidated.

The leaf edit appends a new definition to `Ackermann.qdt`, a file that no other file imports. The `declarationIndex` query for that file is invalidated (a new name appears), and the new definition is elaborated, but no other file's queries are affected. The hub edit appends a definition to `Nat.qdt`, which is imported by five other files. Despite the wider dependency fan-out, the rebuild takes the same time: the new definition does not change any existing constant's type or body, so the `constant` queries for existing Nat definitions hash the same, and early cutoff prevents the cascade from propagating to dependent files.

4.7 Discussion

The incremental results demonstrate that the query-based architecture achieves its goal: after an edit, re-elaboration time is proportional to the amount of work that actually changed, not the size of the file or the number of dependents. The 6–8× speedup over cold build on the standard library reflects the fact that most queries are unchanged after a typical edit.

The scaling benchmarks show that qdt's per-definition elaboration cost is competitive with Lean 4 (which additionally performs kernel checking). The conversion checker scales linearly on Church-encoded benchmarks, with a constant-factor gap from the defunctionalised closure representation.

The formal verification ensures that incremental results are provably equivalent to batch evaluation. The free theorem bridges the gap between the memoising build systems and the specification, with all proof obligations discharged except the parametricity axiom.

Chapter 5

Conclusion

All original aims were met. The elaborator supports the full intended type theory — dependent function types, Tarski-style universes with universe polymorphism, and inductive types with recursors — and successfully elaborates a 2,200-line standard library. The build system framework is formalised in Lean 4 with machine-checked correctness proofs: three build systems inhabit the same verified `Build` type. Incremental re-elaboration avoids redundant work, and early cutoff prevents re-elaboration when a definition's type is unchanged.

Several extensions were completed beyond the original proposal. The proposal specified using the Salsa framework as a black box; instead, the build system was formalised from scratch with machine-checked correctness proofs — a contribution that stands independently of the elaborator. The elaborator implements glued evaluation and approximate conversion checking (rigid/flex/full), which were not planned but turned out to be essential for reducing unnecessary dependencies in the query graph. A language server with diagnostics and hover information was built, providing a practical interface for interactive use. Structures with projections and eta-expansion were added to support the standard library's algebraic hierarchy.

TODO: summarise evaluation results.

5.1 Reflections

5.1.1 Choosing Lean 4

The project was initially planned in Rust (using the Salsa framework), initially implemented in OCaml, and finally settled on Lean 4. Each transition was costly but the final choice was the right one. Lean's dependent types made intrinsically scoped terms (`VTm n`), dependently-typed query results (`Val : Key → Type`), and proof-carrying build systems (`{ r // r = compute ... }`) natural to express. The build system correctness proofs live in the same language as the implementation, so the types that the elaborator manipulates are the same objects the proofs reason about. This would have required a separate formalisation in any other language.

5.1.2 Tarski-style universes

The decision to use Tarski-style rather than Russell-style universes simplified the core but complicated the surface language. Every type annotation requires explicit `El` coercions in the core, and the user-facing syntax must hide them. In hindsight, the cleaner metatheory was worth it — the syntactic separation between types and terms made the evaluator and conversion checker easier to get right — but a Russell-style system with a thin elaboration layer on top might have been equally viable.

5.1.3 Incrementality without metavariables

Omitting metavariables and unification was a deliberate scoping decision, but it also turned out to be architecturally convenient. Unification introduces global side effects: solving a metavariable in one definition can affect the elaboration of another. This would complicate the dependency tracking that makes incrementality correct. The fully explicit surface language is verbose, but it makes each declaration's elaboration self-contained — exactly the property the build system needs.

5.2 Future work

- **Parallelism.** Mokhov et al. [9] refine the BSALC framework with *selective functors*, which sit between applicative and monadic tasks. Applicative tasks have statically known dependencies and can be trivially parallelised; monadic tasks have dynamic dependencies and must be sequential. Selective tasks can speculatively execute both branches of a conditional in parallel, discarding the unused one. Incorporating selective tasks would allow the build system to parallelise queries with static dependencies (e.g. parsing independent files) while retaining dynamic dependencies where needed (e.g. elaboration that discovers references at runtime).
- **Cancellation.** A language server should cancel in-progress elaboration when the user edits again. A middleware framework could support this via an exception monad that preserves partial progress in the memo store.
- **Pattern matching.** The core theory uses recursors directly. A pattern-matching compiler translating case trees to recursor applications would make the surface language more practical without changing the core.

Bibliography

[1] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.

[2] U. Norell, "Dependently Typed Programming in Agda," in *Advanced Functional Programming*, Springer, 2009, pp. 230–266. doi: 10.1007/978-3-642-04652-0_5.

[3] B. Barras *et al.*, "The Coq Proof Assistant Reference Manual," 1999.

[4] A. Kladov and contributors, "rust-analyzer." [Online]. Available: https://rust-analyzer.github.io/

[5] N. Matsakis and contributors, "Salsa: A generic framework for on-demand, incrementalized computation." [Online]. Available: https://github.com/salsa-rs/salsa

[6] O. Fredriksson, "sixty." [Online]. Available: https://github.com/ollef/sixty

[7] U. A. Acar, G. E. Blelloch, and R. Harper, "Adaptive Functional Programming," in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2002, pp. 247–259.

[8] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, Demand-Driven Incremental Computation," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2014, pp. 156–166.

[9] A. Mokhov, N. Mitchell, and S. P. Jones, "Build Systems à la Carte: Theory and Practice," *Journal of Functional Programming*, vol. 30, p. e11, 2020.

[10] J. Dunfield and N. Krishnaswami, "Bidirectional Typing," *ACM Computing Surveys*, vol. 54, no. 5, 2021, doi: 10.1145/3450952.

[11] A. Abel, "Normalization by Evaluation: Dependent Types and Impredicativity," *Lecture notes*. 2013.

[12] A. Kovács, "smalltt." [Online]. Available: https://github.com/AndrasKovacs/smalltt

[13] V. R. Pratt, "Top Down Operator Precedence," *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pp. 41–51, 1973.

[14] Microsoft, "Roslyn: The .NET Compiler Platform." 2015.

[15] N. G. de Bruijn, "Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem," *Indagationes Mathematicae*, vol. 75, no. 5, pp. 381–392, 1972.

[16] P. Dybjer, "Inductive Families," in *Formal Aspects of Computing*, Springer, 1994, pp. 440–465.

[17] J. C. Reynolds, "Types, Abstraction and Parametric Polymorphism," in *Information Processing 83*, Elsevier, 1983, pp. 513–523.

[18] P. Wadler, "Theorems for Free!," in *Proceedings of the 4th International Conference on Functional Programming Languages and Computer Architecture (FPCA)*, ACM, 1989, pp. 347–359.

[19] J. Voigtländer, "Free Theorems Involving Type Constructor Classes," in *Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, ACM, 2009, pp. 173–184.

[20] R. Atkey, "Relational Parametricity for Higher Kinds," in *Proceedings of the 21st EACSL Annual Conference on Computer Science Logic (CSL)*, Schloss Dagstuhl, 2012, pp. 46–61.

Appendices

Appendix A

Project Proposal

Query-based Incremental Elaborator for a Dependently Typed Language

A.0.1 Introduction and Description

This project aims to implement an incremental, query-based elaborator for a dependently typed toy language, for efficient type checking in interactive development environments, such as proof assistants. Traditional type checkers for dependent types operate in batch mode, re-checking entire programs after each change. This project will investigate query-based compilation techniques, building on Olle Fredriksson's work and modern language servers, to enable efficient incremental type checking.

The core objective is to build an elaborator that can efficiently respond to local changes in source code by recomputing only the affected parts of the type checking process. This will be achieved using a query-based architecture where type checking operations are decomposed into memoisable queries with explicit dependencies. The implementation will use the Salsa framework, written in Rust, which provides the tools for incremental on-demand computation by automatically tracking dependencies and memoisation.

A.0.2 Starting Point

No prior work was done before October.

A.0.3 Substance and Structure

1. Core Language: a minimal dependently typed calculus with Pi, universes, and basic inductive types. The syntax will support function definitions, pattern matching, and type annotations.
2. Query-based elaborator: bidirectional type checking decomposed into queries (e.g., `infer_type`, `check_type`, `normalise_term`). Each query can be memoised, with proper dependency tracking to enable incremental recomputation.
3. Incremental infrastructure: use of the Salsa framework to handle source file changes, manage parse trees, and dependency graphs for queries. Detection of changes and lazy memoisation strategies. This is the most important component of the project.

4. Performance evaluation framework: benchmarking suite to measure incremental performance against batch re-elaboration, including synthetic workloads simulating typical editing patterns.

Key algorithms include type unification, bidirectional type checking with constraint generation, normalisation for dependent types.

A.0.4 Success Criteria and Evaluation Plan

A.0.4.1 Success Criteria

1. Correctly type check a language with dependent types, including Pi types, Sigma types and inductive types
2. Demonstrate measurable performance improvement for incremental changes vs full re-elaboration

A.0.4.2 Evaluation Plan

- Correctness: test suite of programs including simple proofs of equality, existential and universal quantification (dependent pair and arrow types).
- Performance and scalability: Benchmarks measuring re-elaboration time for:
  - local definition changes
  - type signature modifications
  - cascading dependency updates

  on sample programs, generated by a computer, of at least 1000 lines of code

A.0.5 Work Plan

A.0.5.1 Michaelmas Term

- 9 Oct – 22 Oct: Read Salsa documentation, study dependent type implementation, implement toy examples, design language syntax and semantics
- 23 Oct – 6 Nov: Implement elaborator for dependent types over a query-based interface
- 7 Nov – 19 Nov: Complete elaborator for dependent types, including identity types, dependent pairs; write example programs
- 20 Nov – 3 Dec: Decompose elaborator, and other stages of compilation process into fine-grained queries

A.0.5.2 Lent Term

- 22 Jan – 4 Feb: Implement incrementalisation with change detection, cache invalidation, and dependency tracking, utilising Salsa
- 5 Feb – 18 Feb: Complete incrementalisation, with source-based tracking
- 19 Feb – 4 Mar: Optimise incrementalisation, query granularity and dependency tracking
- 5 Mar – 18 Mar: Evaluation and performance benchmarks, with synthetic programs

A.0.5.3 Easter Term

- 30 Apr – 15 May: Dissertation

A.0.6 Resources Declaration

- Personal laptop (32 GB RAM, Intel Ultra 7 155) for development

- Backup strategy: Git repository on GitHub
- Contingency plan: Backup laptop available

A.0.7 Ethics Approval

Not required as no human participants or personal data involved.